

POCKETSMART AI-BUDGET PLANNER

AN AI-POWERED SMART BUDGET PLANNER FRAMEWORK

A PHASE-WISE COMPREHENSIVE ARCHITECTURAL SYSTEM DOCUMENTATION

PHASE 1: BRAINSTORMING & IDEATION PHASE

- **Core Problem Statement:** Budgeting for lifestyle events like home interior modifications, wedding events, and jewelry procurement is usually scattered across fragmented offline templates or manual web searches, causing heavy budget overruns and operational human calculation errors.
- **The AI Innovation:** PocketSmart AI acts as an autonomous digital financial coordinator that captures structural limits (budgets, counts) and automatically splits them into optimized, micro-allocated categories.
- **Ecosystem Integration:** Embedded direct retail deep-links (Amazon, Flipkart, Swiggy, Zomato, CaratLane) within the output grid views to eliminate manual tracking.

PHASE 2: REQUIREMENT ANALYSIS PHASE

2.1 Functional System Specifications

- **UC-001 (Secure Session Init):** System must handle custom identity validation strings against verified database memory records to lock interface sessions.
- **UC-002 (Dynamic Context Dispatching):** Application dashboard must seamlessly transfer context configurations across Home, Party, and Jewelry pipelines via strict HTTP address parameters.
- **UC-003 (Autonomous Array Evaluation):** Client view script layers must immediately process custom resource counts and print exact transaction matrices on form execution events.

2.2 Non-Functional Compliance Standards

- **Performance Footprint:** System routing lookups and backend response streams must execute in under 400ms under standard loopback interface connection boundaries.

- **Viewport Responsiveness:** Interface containers must utilize fluid flex-box grids to maintain consistent render layouts on mobile viewports.

PHASE 3: PROJECT DESIGN PHASE

The system is organized into a clean decoupled Client-Server layout matrix. The client layers interact natively via client-side scripts to maintain low latency profiles, bypassing continuous backend server round-trip connection overheads for mathematical operations.

- **Base Canvas Theme:** Anti-glare slate-blue viewport theme (#ebf2f7).
- **Utility Option Boxes:** Crisp white card boxes styled with micro-shadowing (box-shadow: 0 4px 15px rgba(0,0,0,0.05)).
- **Primary Action Accents:** Official Deep Indigo Blue borders (#1a4f8a) paired with tactical secondary accents like Amber Gold (#f39c12) for chronological history tools.

PHASE 4: PROJECT PLANNING PHASE

The deployment lifecycle followed an agile micro-sprint planning paradigm designed to deliver production modules within strict timelines:

Sprint Identifier	Operational Task Mandate	Assigned Milestone Deliverable
Sprint Lifecycle 01	Environment assembly, microserver configuration, app.py routing setup.	Base Core Registry Operational
Sprint Lifecycle 02	UI development for login, home_planner, party_planner templates.	Frontend Forms Deployed
Sprint Lifecycle 03	CORS configuration setup, cache validation error fixes, link auditing.	Master Application Complete

PHASE 5: PROJECT DEVELOPMENT PHASE

5.1 Master Backend Microserver Architecture (app.py)

Below is the complete production source code engineered inside the core backend engine layer to initialize routing maps across all software components:

```
import os
import json
import urllib.parse
from fastapi import FastAPI, HTTPException, Form
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI(title="PocketSmart: AI Budget Planner Master Engine")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
```

```
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

registered_users = ["sai", "admin", "user123", "aswathi", "sumathi",
"sankari"]

@app.get("/dashboard", response_class=HTMLResponse)
async def serve_dashboard(): return load_html_page("dashboard.html")
```

PHASE 6: PROJECT TESTING PHASE

- **Test Case 01 (Server Syntax Blockages):** Caught a severe `SyntaxError` issue caused by HTML single quotes breaking python multi-line definitions. Fixed by decoupling page assets completely via unified template file streams (`load_html_page`).
- **Test Case 02 (Cross-Origin Blockages):** Solved a 'Connection Error!' warning caused by server cross-origin protection rules. Resolved by mapping active client routing layers directly inside individual frontend containers.

PHASE 7: PROJECT DOCUMENTATION PHASE

The engineering documentation phase established an immutable set of setup resource matrices, configuration maps, and runtime testing instructions saved inside a clean `README.md` archive. This guarantees that any evaluator can effortlessly spawn the multi-module service architecture on any destination operating registry using basic initialization guidelines (`python app.py`).

PHASE 8: PROJECT DEMONSTRATION PHASE

The application stack was fully validated by executing end-to-end workflow evaluations across every operational viewport container:

1. The client signs in via the secure credential input layer screen interface.
2. The application passes URL parameter elements safely to inject custom user identities across the master navigation matrix container view dashboard.
3. The user successfully enters the Home, Party, and Jewelry planner sub-screens to evaluate fiscal division plans with functional external shopping redirects.